

ISA 18 BITS – Instruções – GRR20256088

opcode	nome	instrução	funcionalidade
0000	load word	lw rd, rs1, imm	carrega em rd o valor da memória no endereço rs1 + imm
0001	store word	sw rd, rs1, imm	armazena o valor de rd na memória no endereço rs1 + imm
0010	add immediate	addi rd, rs1, imm	soma rs1 com valor imediato e guarda em rd
0011	jump and link	jal rd, imm	salva o endereço de retorno em rd e salta para o endereço em imm
0100	jump and register	jalr rd, rs1, imm	salva o endereço de retorno em rd e salta para rs1 + imm
0101	branch if equal	beq rs1, rs2, imm	se rs1 == rs2, altera o pc para o endereço pc = imm
0110	branch if greater or equal	bge rs1, rs2, imm	se rs1 >= rs2, altera o pc para o endereço pc = imm
0111	branch if not equal	bne rs1, rs2, imm	se rs1 <> rs2, altera o pc para o endereço pc = imm
1000	add	add rd, rs1, rs2	soma rs1 e rs2, armazena em rd
1001	and	and rd, rs1, rs2	faz operação lógica and bit a bit entre rs1 e rs2
1010	or	or rd, rs1, rs2	faz operação lógica or bit a bit entre rs1 e rs2
1011	fused multiply-add	fma rd, rs1, rs2, rd	multiplica rs1 por rs2 e soma com o valor atual de rd
1100	shift right arithmetic	sra rd, rs1, rs2	desloca rs1 para a direita em rs2 bits mantendo o sinal (divide por potência de 2)
1101	shift left logic	sll rd, rs1, rs2	desloca rs1 para a esquerda em rs2 bits (multiplica por potência de 2)
1110	xor	xor rd, rs1, rs2	faz operação lógica xor bit a bit entre rs1 e rs2
1111	sub	sub rd, rs1, rs2	subtrai rs1 e rs2, armazena em rd

Registadores

registadores	abi	definição
r0	x0	registro nulo
r1	a0	parâmetro da função
r2	a1	parâmetro da função
r3	t0	temporário
r4	t1	temporário
r5	t2	temporário
r6	sp	ponteiro da stack
r7	ra	endereço de retorno

Formatação das instruções

Tipo	4 bits	3 bits	3 bits	3 bits	5 bits
R	opcode	rd	rs1	rs2	sem uso
I	opcode	rd	rs1	imm	
B	opcode	rs2	rs1	imm(endereço)	
J	opcode	rd	imm(endereço)		

Unidades de controle

tipo	inst.	opcode	pccontrol	wbreg	mux_ula	controlado	wm	opcodeula	controlers2rd	hex
I	lw	0000	00	1	1	01	0	000	0	120
I	sw	0001	00	0	1	00	1	000	0	11
I	addi	0010	00	1	1	00	0	000	0	180
J	jal	0011	01	1	0	10	0	000	0	340
J	jalr	0100	01	1	0	10	0	000	0	340
B	beq	0101	10	0	0	00	0	000	1	401
B	bge	0110	10	0	0	00	0	000	1	401
B	bne	0111	10	0	0	00	0	000	1	401
R	add	1000	00	1	0	00	0	000	0	100
R	and	1001	00	1	0	00	0	001	0	102
R	or	1010	00	1	0	00	0	010	0	104
R	fma	1011	00	1	0	00	0	011	0	106
R	sra	1100	00	1	0	00	0	100	0	108
R	sll	1101	00	1	0	00	0	101	0	10a
R	xor	1110	00	1	0	00	0	110	0	10c
R	sub	1111	00	1	0	00	0	111	0	10e

* o número hexadecimal é atribuído somente às unidades de controle, ou seja, com exceção do opcode.

Nomenclatura

opcode: opcode das instruções.

pccontrol: controla o multiplexador responsável pelo aumento do program counter (PC).

wbreg: escrita do banco de registradores.

mux_ula: controla o multiplexador que gerencia imediatos e valores de registradores para a ULA.

controlado: controla o multiplexador que seleciona o que será escrito no dado do banco de registradores.

wm: escrita da memória de dados.

opcodeula: opcode das instruções aritméticas.

controlers2rd: controla o multiplexador que administra se será inserido o rs2 ou o próprio rd na entrada rd do banco de registradores.

Componentes utilizados

PC(Program counter): 1 registrador de 18 bits.

Banco de registradores: 8 registradores de 18 bits.

Memória de instrução: 1 memória ROM.

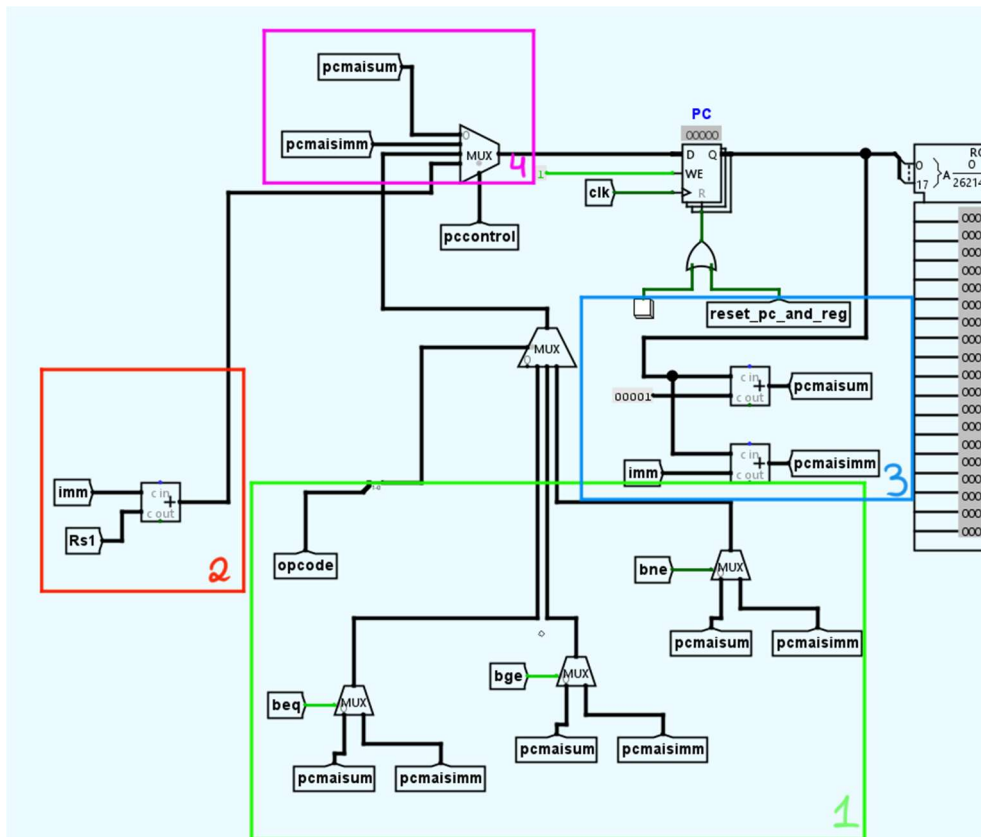
Memória de dados: 1 memória RAM.

Memória de controle: 1 memória ROM.

Quantidades de plexers: 20 (1 demux e 19 mux).

Decisões de implementação

1) Program Counter:



Área 1, responsável pelos branches:

Aqui, o papel do multiplexador grandão (de 4 entradas) é controlar os mini multiplexadores (de 2 entradas) por meio do opcode. Sabemos pela tabela da página anterior que:

opcode do beq: 0101;

opcode do bge: 0110;

opcode do bne: 0111;

Ou seja, caso o opcode seja 0101 (beq), o splitter passará pelo fio de controle do mux somente os 2 bits menos significativos, isto é, 0 e 1. Consequentemente, 01 deixará a primeira entrada passar, que será a saída do mini mux que controla o novo endereço do PC dependendo do sinal do túnel beq.

Então, se beq for 1, o que significaria que os dois valores são iguais, o program counter seria direcionado para o endereço atual mais o imediato fornecido. A mesma lógica vale para os outros branches.

Área 2, 3 e 4, cálculos gerais:

Essas não têm muito segredo por trás, apenas calculam o endereço que for preciso para o PC dependendo da instrução dada.

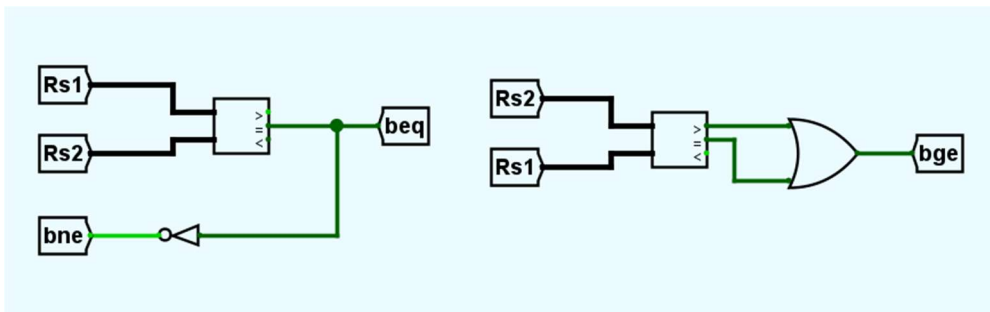
A área 2, calcula $rs1 + imm$, cuja soma resultante é usada somente pelo jalr, o qual realiza um salto para o endereço da soma obtida.

Já a área 4 é bem descritiva: uma entrada calcula o valor do endereço atual de PC com a constante 1, e a outra só muda a constante pelo valor imediato.

Na área 3, analisa-se o cálculo das operações $(pc+1)$ e $(pc+imm)$ utilizadas através de túneis pelas áreas 4 e 1.

2) Branches:

Aproveitando a citação no tópico anterior, temos as lógicas simples das 3 branches utilizadas na ISA:



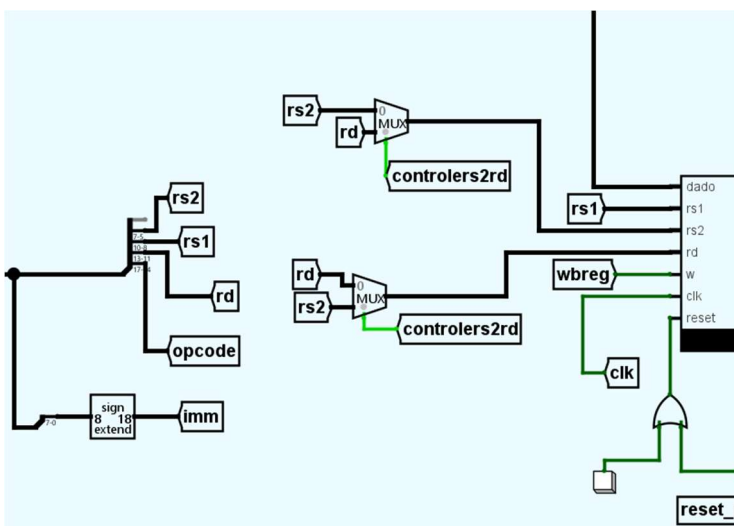
Com os comparadores do logisim, podemos comparar os valores presentes nos registradores e saber qual a relação dos mesmos:

beq será 1 se rs1 e rs2 forem iguais;

bge será 1 se rs1 e rs2 forem iguais ou se rs1 for maior que rs2;

Como bne é o contrário de beq, basta aplicar uma porta NOT na saída do comparador de beq, que fará sinal 1 se os valores não forem iguais. Consequentemente, os valores serão encaminhados para os plexers do PC, que foram explicados na sessão anterior.

3) Das instruções para o banco de registradores:



Na saída da memória de instruções:

Antes do splitter, o sign extender coleta os 8 bits do valor imediato, o qual seleciona estes e os estende para 18 bits, mantendo seu sinal. Dessa forma, permitimos com que a ISA possa operar utilizando valores negativos, já que os sinais poderão ser identificados com complemento de 2.

No splitter, a instrução de 18 bits é separada conforme tabela de formatação das instruções, então:

bit + significativo -> bit – significativo

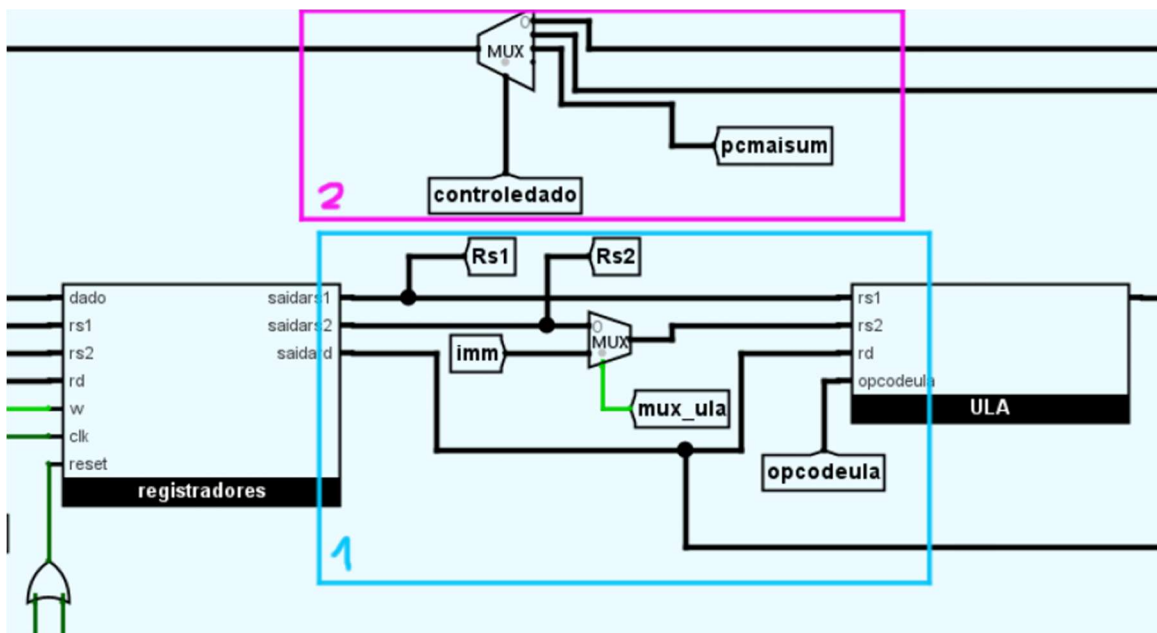
opcode(4) | rd(3) | rs1 (3) | rs2 (3) | sem uso(ou imm dependendo da instrução)

Na entrada do banco de registradores:

Após essa separação dos bits, os fios se dirigem em suas saídas específicas e nomeadas normalmente, com exceção de rs2 e rd, que passam por um mux antes de se direcionarem para os registradores.

Esses plexers servem para administrar instruções diferentes, visto que muitas delas não tem registradores em comum que estejam na mesma ordem, como o tipo B e tipo R.

4) ULA e Registradores



* Aqui já tem bastante coisa, por isso separei em quadradinhos novamente para facilitar

Área 1:

Rs1 e Rs2 são direcionadas normalmente, enquanto o mux do fio da saidars2 serve para controlar se deve ou não entrar um valor imediato em rs2, conforme a instrução necessária.

Área 2:

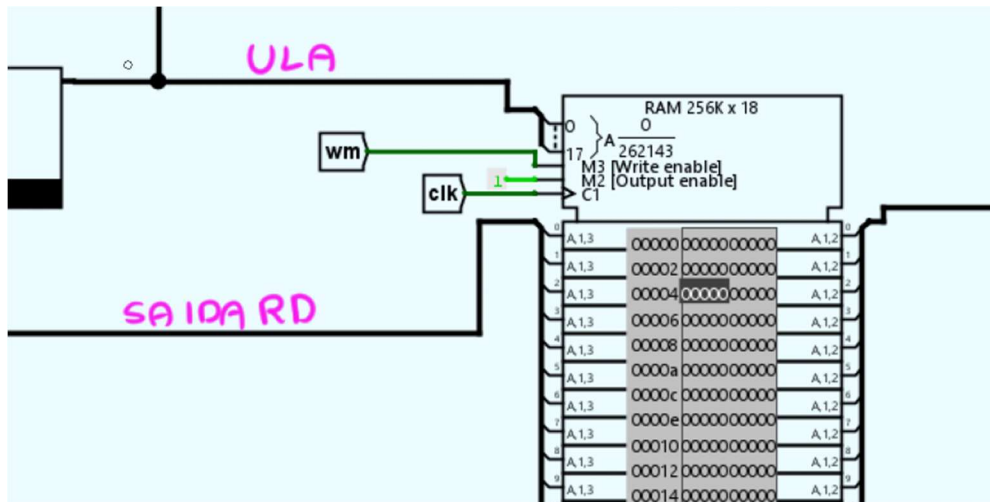
Esse mux controla qual tipo de dado será escrito no registrador escolhido, com os seguintes bits de controle:

00: resultado da ULA;

01: valor da memória RAM de dados;

10: PC + 1, para jumps.

5) Memória de Dados

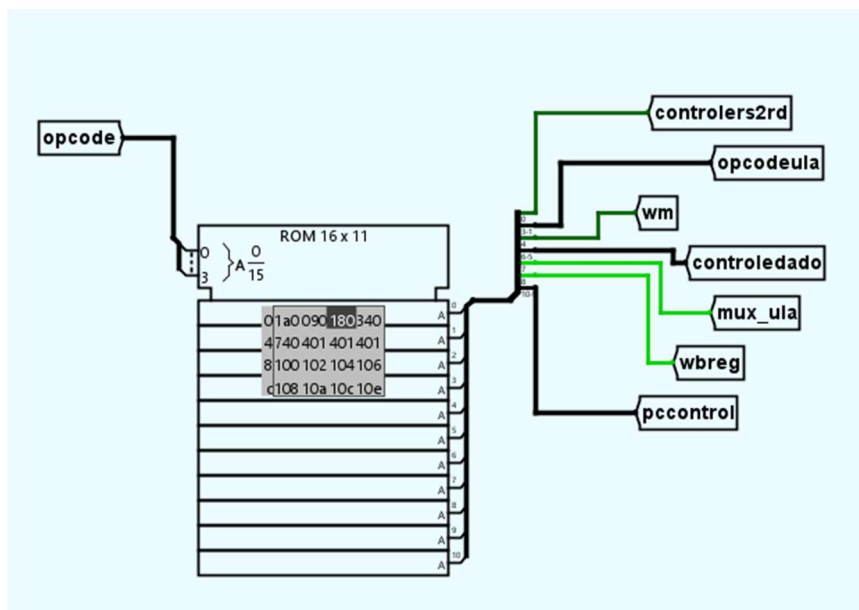


Esta é responsável pelo load e store das words, portanto:

lw rd, rs1, imm = significa que o valor que está alocado no endereço [rs1 + imm] será armazenado em rd. O endereço é calculado através da ULA, por isso esta é direcionada pela entrada da RAM.

sw rd, rs1, imm = aloca o valor presente em rd no endereço [rs1 + imm], o qual também será calculado através da ULA.

6) Memória de controle



Por último, mas não menos importante, temos a memória de controle, que é bem simples também:

Como ela gerencia as unidades de controle,

Como ela controla as operações da ISA, usa-se um splitter para distribuir cada bit das unidades de controle, e, conseqüentemente, é indexado os valores hexadecimais presentes nas tabelas já apresentadas.

Tradução dos códigos

Formato das traduções, com numeração de linha:

1) comentário(#), 2) assembly, 3) linguagem de máquina, 4) hexadecimal da linguagem binária

* a linguagem de máquina possui espaços que ajudam na visualização dos registradores, opcode e imediatos.

Todas as instruções (em ordem) :

r1 recebe 4 (hex: 00004)

addi r1, r0, 4

0010 010 000 00000100

8804

r2 recebe 5 (hex: 00005)

addi r2, r0, 5

0010 010 000 00000101

9005

r1 recebe $r1 + r1 = 8$ (hex: 00008)

add r1, r1, r1

1000 001 001 001 00000

20920

r2 recebe $r2 + r2 = 10$ (hex: 0000a)

add r2, r2, r2

1000 010 010 010 00000

21240

verifica se r1 e r2 são iguais. É falso, PC recebe $PC + 1$

beq r1, r2, 4

0101 001 010 00000100

14a04

verifica se $r2 \geq r1$. É verdade, PC recebe $PC + 6$

bge r2, r1, 6

0110 010 001 00000110

19106

verifica se $r2 \neq r1$. É verdade, PC recebe $PC + 3$

bne r2, r1, 3

0111 010 001 00000011

1d103

r3 recebe 8 (hex: 00008)

addi r3, r0, 8

0010 011 000 00001000

9808

r3 recebe $r1 * r2 + r3 = 88$ (hex: 000058)

fma r3, r1, r2

1011 011 001 010 00000

2d940

r3 recebe $r1 \text{ and } r2 = 8$ (hex: 00008)

and r3, r1, r2

1001 011 001 010 00000

25940

r3 recebe $r1 \text{ or } r2 = 10$ (hex: 0000a)

or r3, r1, r2

1010 011 001 010 00000

29940

r3 recebe $r1 \text{ xor } r2 = 2$ (hex: 00002)

xor r3, r1, r2

1110 011 001 010 00000

39940

r3 recebe $r2 - r1 = 2$ (hex: 00002) – valor ficou o mesmo da instrução anterior -

sub r3, r2, r1

1111 011 010 001 00000

3da20

r1 recebe 2 (hex: 00002)

addi r1, r0, 2

0010 001 000 00000010

8802

verifica se $r1 = r3$. É verdade, PC recebe $PC + 2$

beq r1, r3, 2

0101 001 011 00000010

14b02

verifica se $r1 \geq r2$. É falso, PC recebe $PC + 1$

bge r1, r2, 6

0110 001 010 00000110

18a06

verifica se $r3 \neq r1$. É falso, PC recebe $PC + 1$

bne r3, r1, 3

0111 011 001 00000011

1d903

r3 recebe $r2 / 4$ (parte inteira) = 2 (hex: 00002) – valor ficou o mesmo da instrução anterior –

sra r3, r2, r1

1100 011 010 001 00000

31a20

r3 recebe $r2 * 4 = 40$ (hex: 000028)

sll r3, r2, r1

1101 011 010 001 00000

35a20

PC recebe PC + 3 e salva próxima instrução em r1

jal r1, 3

0011 001 00000000011

c803

PC recebe PC + 12 e salva próxima instrução em r2

jalr r2, r1, 12

0100 010 001 00001100

1110c

o valor de r2 é alocado em &[r1 + imm]

sw r2, r1, 1

0001 010 001 00000001

5101

r1 recebe o valor de &[r1 + imm]

lw r1, r1, 1

0000 001 001 00000001

901

Programa completo com todas as instruções:

Assembly

addi r1, r0, 4

addi r2, r0, 5

add r1, r1, r1

add r2, r2, r2

beq r1, r2, 4

bge r2, r1, 6

bne r2, r1, 3

addi r3, r0, 8

fma r3, r1, r2

and r3, r1, r2

or r3, r1, r2

```
xor r3, r1, r2
sub r3, r2, r1
addi r1, r0, 2
beq r1, r3, 2
bge r1, r2, 6
bne r3, r1, 3
sra r3, r2, r1
sll r3, r2, r1
jal r1, 3
jal r2, r1, 12
sw r2, r1, 1
lw r1, r1, 1
```

Linguagem de máquina:

```
0010 010 000 00000100
0010 010 000 00000101
1000 001 001 001 00000
1000 010 010 010 00000
0101 001 010 00000100
0110 010 001 00000110
0111 010 001 00000011
0010 011 000 00001000
1011 011 001 010 00000
1001 011 001 010 00000
1010 011 001 010 00000
1110 011 001 010 00000
1111 011 010 001 00000
0010 001 000 00000010
0101 001 011 00000010
0110 001 010 00000110
0111 011 001 00000011
1100 011 010 001 00000
1101 011 010 001 00000
0011 001 00000000011
```

0100 010 001 00001100
0001 010 001 00000001
0000 001 001 00000001

Tradução do código do github:

```
void main(){
    int A[10], B[10], C[10], i;
    for(i=0; i<10; i++){
        A[i] = i;
        B[i] = 10-i;
    }
    for(i=0; i<10; i++)
        C[i] = A[i] + B[i];
}
```

Assembly:

addi sp, sp, -30

ponteiros para inicialização dos vetores

addi t0, sp, 0; A[0]

addi t1, sp, 10; B[0]

addi t2, sp, 20; C[0]

a0 recebe i, e a1 recebe 10

addi a0, r0, 0

addi a1, r0, 10

for1:

bge a0, a1, forafor1

preenchimento dos vetores

sw a0, t0, 0; o que está em a0(i) vai para t0(A[i])

sub a1, a1, a0; sobrescreve a1

sw a1, t1, 0; o que está em a1(10 – i) vai para t1(B[i])

addi a1, r0, 10; restaura a1

incrementa o i e os ponteiros dos vetores

addi t0, t0, 1

```
addi t1, t1, 1
addi a0, a0, 1
jal r0, for1
```

forafor1:

reinicializa tudo para outro laço

```
addi t0, sp, 0
addi t1, sp, 10
addi t2, sp, 20
addi a0, r0, 0
addi a1, r0, 10
```

for2:

bge a0, a1, fim

carrega A[i] em a0

lw a0, t0, 0; o a0(i) vai receber o valor armazenado no endereço de t0 + 0 (a[i])

carrega B[i] em a1

lw a1, t1, 0; o a1(antigo limite – vai ser sobrescrevido por enquanto -) recebe o valor no endereço de t1 + 0 (b[i])

soma

add a0, a0, a1; a0 recebe a0(A[i]) + a1(B[i])

sw a0, t2, 0; C[i] recebe a0 (A[i] + B[i])

incrementos dos ponteiros

```
addi t0, t0, 1
addi t1, t1, 1
addi t2, t2, 1
```

a0 (i) é incrementado 1 vez, subtraindo a0 de a1(b[i]) e somando com 1

```
sub a0, a0, a1
addi a0, a0, 1
```

addi a1, r0, 10

jal r0, for2

fim:

addi sp, sp, 30; desaloca memória

ebreak

Linguagem de Máquina:

0010 011 110 00000000

0010 100 110 00001010

0010 101 110 00010100

0010 001 000 00000000

0010 010 000 00001010

0110 001 010 00001111

0001 001 011 00000000

1111 010 010 001 00000

0001 010 100 00000000

0010 010 000 00001010

0010 011 011 00000001

0010 100 100 00000001

0010 001 001 00000001

0011 00000000000110

0010 011 110 00000000

0010 100 110 00001010

0010 101 110 00010100

0010 001 000 00000000

0010 010 000 00001010

0110 001 010 00100000

0000 001 011 00000000

0000 010 100 00000000

1000 001 001 010 00000

0001 001 101 00000000

0010 011 011 00000001

0010 100 100 00000001

0010 101 101 00000001

1111 001 001 010 00000

0010 001 001 00000001

0010 010 000 00001010

0011 00000000010100

0010 110 110 00011110